CAHIER DES CHARGES

Polymarket Strategy Research Lab

Laboratoire local d'analyse et de simulation de stratégies de trading

Version 0.1 • 26 octobre 2026 • Windows local • Paper trading uniquement

Objectif : évaluer si des comportements observés chez trois traders performants sur les marchés crypto de Polymarket peuvent être reproduits de manière crédible, après coûts d'exécution et validation hors échantillon.

Document de cadrage initial. Les choix techniques et les règles de frais devront être vérifiés et versionnés lors de l'implémentation.

1. Résumé exécutif

Le projet consiste à construire un environnement local de recherche : collecte de données publiques, analyse des comportements de traders, transformation des hypothèses en règles exécutables, paper trading, comparaison avec des benchmarks indépendants et visualisation des résultats.

La V1 se limite à trois traders, aux marchés crypto et aux ordres taker. Les exécutions seront estimées à partir de la profondeur observable, en intégrant un modèle explicite de latence, de vieillissement du carnet, de spread, de slippage et de frais. Les ordres maker seront ajoutés dans une version ultérieure.

Règle centrale : aucun ordre réel ne sera envoyé. Les résultats seront des estimations de simulation et ne constitueront pas une garantie de rentabilité future.

2. Objectifs et questions de recherche

- Identifier trois traders présentant un historique public pertinent et documenter les critères de sélection.
- Analyser leurs comportements sur les marchés crypto à court terme sans confondre observation et hypothèse.
- Tester des stratégies candidates dans des conditions chronologiques réalistes.
- Mesurer le résultat après coûts, drawdown, concentration des gains, exposition et stabilité.
- Comparer chaque stratégie à des benchmarks indépendants.
- Évaluer séparément la validité technique, historique et en direct.

3. Périmètre de la V1

Inclus : collecte de données publiques, stockage local, suivi de trois traders, rapports d'analyse, stratégies versionnées, simulateur taker, portefeuille virtuel, dashboard, journaux et tests.

Exclus : argent réel, ordres maker, autres catégories de marchés, exécution automatisée réelle et déploiement VPS comme environnement principal.

Capitaux virtuels à comparer : 100, 250, 500, 1 000, 2 500, 5 000 et 10 000 USD/USDC. Ces montants sont des scénarios de test configurables et non des recommandations d'investissement.

4. Architecture fonctionnelle

Module	Responsabilité principale
1. Collecte	Récupérer transactions, marchés, activité publique et flux de carnet disponibles.
2. Qualité des données	Normaliser, dédupliquer, détecter les trous et signaler les données périmées.
3. Stockage local	Conserver événements bruts, historiques, snapshots, configurations et résultats.
4. Analyse trader	Produire faits, hypothèses, inconnues et expériences à tester.
5. Registre de stratégies	Versionner règles, paramètres, périodes et hypothèses.
6. Simulation	Estimer les exécutions taker, les coûts, les fills partiels et les rejets.
7. Grand livre / métriques	Calculer cash, positions, PnL, drawdown, exposition et frais.
8. Benchmark indépendant	Exécuter les contrôles sur les mêmes données et conventions.
9. Dashboard	Visualiser performance, positions, carnet, coûts, qualité et comparaison.
10. Validation	Tester calculs, biais temporels, robustesse et reproductibilité.

5. Architecture technique proposée

- Python pour collecte, simulation et analyse.
- asyncio et WebSocket pour les flux de marché en temps réel.
- SQLite pour un prototype local ; PostgreSQL/TimescaleDB si le volume le justifie.
- pandas, NumPy et SciPy pour les analyses ; pytest pour les tests.
- Streamlit et Plotly pour un dashboard local rapide à développer.
- Git et configurations versionnées pour assurer la reproductibilité.

L'architecture doit rester modulaire sans imposer d'emblée une infrastructure distribuée. Les endpoints, formats et barèmes de frais seront vérifiés dans la documentation officielle au moment du développement.

6. Règles du simulateur taker

Lorsqu'un bot décide d'acheter, le simulateur consomme les niveaux ask disponibles du meilleur au moins bon prix. Pour une vente, il consomme les niveaux bid du meilleur au moins bon prix. Le prix moyen est pondéré par les quantités exécutées. La quantité demandée peut être partiellement exécutée ou rejetée si la profondeur est insuffisante.

- Enregistrer le timestamp source, l'heure de réception, la décision et l'heure simulée d'exécution.
- Modéliser la latence et vérifier l'âge du snapshot du carnet.
- Séparer spread, slippage, frais et coût de profondeur, en évitant leur double comptage.
- Appliquer les règles de frais vérifiées pour le marché et la période ; ne pas utiliser un taux universel non confirmé.
- Conserver la quantité demandée, exécutée et restante, les niveaux consommés et les hypothèses utilisées.
- Valoriser les positions ouvertes à partir d'un prix de liquidation estimé, pas uniquement du mid-price.

La V1 ne simule pas les ordres maker. Leur extension devra prendre en compte la file d'attente, les fills partiels, les annulations et le risque d'adverse selection.

7. Validation technique

- Tests unitaires de consommation du carnet, prix moyen pondéré, frais et PnL.
- Tests comptables : cash, positions, frais et valeur liquidative doivent se réconcilier.
- Tests de données périmées, déconnexions, doublons et événements hors ordre.
- Tests anti-look-ahead : aucune information future ne peut influencer une décision passée.
- Tests d'idempotence : un fill ne peut être compté deux fois.
- Tests de limites : le bot ne peut dépenser plus que son capital virtuel ni dépasser les limites de position.

8. Validation historique

Séparer chronologiquement les périodes de recherche, validation et test final. Les paramètres sont choisis sur la période de recherche, comparés sur la validation et évalués une seule fois sur le test final autant que possible. Si le carnet historique n'est pas disponible, les résultats d'exécution historiques doivent être qualifiés d'approximatifs.

- Rejouer les événements dans l'ordre chronologique.
- Appliquer latence et coûts sans utiliser le futur.
- Rapporter PnL net, drawdown, profit factor, taux de réussite avec gains/pertes moyens, frais, slippage, exposition et concentration des gains.
- Conserver la liste de toutes les variantes testées pour rendre visible le risque de surapprentissage.

9. Validation en direct

Les bots observent les marchés en temps réel mais restent en paper trading. Le système compare les prix du trader, le prix reçu par le collecteur et le prix estimé exécutable après latence. Il journalise les décisions, rejets, données périmées, fills partiels et écarts de performance.

La durée d'observation doit être définie en fonction du rythme de la stratégie, du nombre de transactions et de l'incertitude statistique. Une courte période positive ne suffit pas à déclarer une stratégie fiable.

10. Benchmark Engine indépendant

Le benchmark est un module distinct des stratégies évaluées. Il utilise les mêmes flux, périodes, hypothèses de capital, limites de risque, conventions de coûts et calculateur de métriques.

- Copie différée du trader : reproduire le signal observé après un délai configurable.
- Contrôle sans signal : comportement de référence simple, documenté et soumis aux mêmes limites.
- Baseline passive : uniquement lorsqu'une comparaison est pertinente pour le marché.
- Contrôle aléatoire contraint : extension possible, avec contraintes comparables et sans accès au futur.

Les benchmarks ne doivent pas être optimisés à partir du test final ni dépendre des décisions internes du bot testé.

11. Métriques du dashboard

- Vue générale : capital virtuel, PnL net, drawdown, positions, nombre de trades et coûts.
- Courbes : capital, PnL cumulé, drawdown, frais et rendement périodique.
- Positions : marché, token, prix moyen, quantité, coût, frais, PnL latent et durée.
- Carnet : bid/ask, spread, profondeur, prix moyen simulé et quantité non exécutée.
- Comparaison : trader original versus bot, délai de copie, écarts de prix et décisions manquées.
- Santé : état des flux, qualité des données, dernières erreurs et statut des validations.

12. Organisation des agents de développement

Le projet peut démarrer avec cinq agents de développement : (1) architecte et contrats, (2) collecte et stockage, (3) simulation et grand livre, (4) analyse trader et stratégies, (5) dashboard. Un agent Validation & Risk peut être ajouté lorsque les interfaces et le premier simulateur sont stables.

Les agents IA proposent des analyses et produisent du code ; les calculs financiers restent déterministes. Chaque agent doit avoir un périmètre, des fichiers possédés, des tests et des critères d'acceptation explicites.

13. Plan de réalisation

Phase	Travail	Livrable / porte de sortie
1. Cadrage	Valider interfaces, données, modèle de frais, schémas et contrats.	Documents validés et dépôt initial.
2. Collecte	Suivre 3 traders crypto et capturer événements/carnets disponibles.	Base locale, journal de qualité et timestamps.
3. Simulateur	Construire exécution taker, latence, frais, fills partiels et grand livre.	Tests comptables et d'exécution verts.
4. Recherche	Analyser les traders et implémenter des stratégies versionnées.	Rapports et premières expériences.
5. Dashboard/live	Paper trading continu et comparaison des bots.	Dashboard et journal d'audit.
6. Évaluation	Benchmarks, stress tests, test hors échantillon et rapport.	Décision documentée par stratégie.

14. Principaux risques et mesures de maîtrise

- **Biais du survivant :** documenter la sélection et, si possible, examiner des profils moins performants comparables.
- **Surapprentissage :** séparer les périodes et conserver les variantes testées.
- **Latence :** mesurer les horodatages et exécuter des scénarios de stress.
- **Frais erronés :** vérifier les règles officielles et dater chaque modèle de frais.
- **Liquidité fantôme :** ne pas supposer que le carnet reste inchangé après le snapshot.
- **Résultats concentrés :** mesurer la contribution des meilleurs trades et la robustesse.
- **Données incomplètes :** signaler les périodes insuffisantes au lieu de les traiter comme complètes.

15. Livrables documentaires

- `PROJECT_SPEC.md` — objectifs et périmètre.
- `ARCHITECTURE.md` — modules et contrats.
- `SIMULATION_RULES.md` — hypothèses d'exécution et comptabilité.
- `VALIDATION_PLAN.md` — tests, périodes, benchmarks et critères d'évaluation.
- `CAHIER_DES_CHARGES.pdf` — version lisible du cadrage.

16. Décisions retenues pour démarrer

- Focus crypto à court terme.
- Trois traders initiaux.
- Windows local pour la première version ; VPS envisagé plus tard si les besoins le justifient.
- Deux niveaux de simulation à terme : commencer simple, puis utiliser la profondeur et les hypothèses de latence.
- Ordres taker uniquement en V1 ; maker en extension.
- Capital virtuel testé de 100 à 10 000 USD/USDC.
- Validation technique, historique et en direct.
- Benchmark indépendant et comparaison systématique.

Fin du cahier des charges — version 0.1. Ce document définit une base de travail ; les choix précis de données, de frais et de périodes devront être validés avant les expériences.